



Sangmyung_Wiki

📅 생성일	2024년 1월 1일 → 2024년 9월 24일
🏷 태그	AWS EC2 CSS Docker MySQL React SpringBoot
📖 한 줄 설명	학교 관련 정보를 공유하는 위키 사이트
👥 개발 인원(역할)	6명 (프론트엔드 3명, 백엔드 2명, 디자인 1명)
📌 관련 활동	상명대학교 졸업 캡스톤 프로젝트
📎 발표 자료	상명위키 발표자료-1.pptx

🔗 Link / Source

GitHub — https://github.com/rilac/SangMyung_WIKI

📖 상세 내용

• 사용대상

상명대학교 재학생 전반. 가입 시 학교 이메일(@sangmyung.kr) 인증을 거쳐야만 회원가입이 완료되는 구조로, 실제 재학생만 글을 작성·수정할 수 있도록 서비스 범위를 제한했다.

• 문제의식

학사 일정, 수강신청 팁, 교양 후기, 학식·셔틀·도서관 이용법 등 학생들에게 꼭 필요한 정보들이 한 곳에 정리되어 있지 않다는 문제에서 출발. 학우들은 매번 단편적으로 흩어진 공지·카톡·커뮤니티 글을 뒤져야 했고, 학사 정보 하나를 찾는 데에도 시간이 걸렸다. "재학생이 직접 쌓고, 재학생이 직접 꺼내 쓰는" 학내 전용 지식 저장소가 필요하다고 판단했다.

- 솔루션재학생 인증을 통과한 사용자만이 위키 문서를 작성·수정할 수 있도록 서비스 경계를 설정하고, 본인의 경험·꿀팁·직접 찾은 정보를 위키 페이지 형태로 축적할 수 있게 했다. 작성된 모든 문서는 가입자 누구나 열람할 수 있으며, 문서 수정 이력(**DocLog**)을 모두 기록해 이전 버전 추적과 기여 내역 관리가 가능하도록 설계했다. 여기에 자유로운 소통을 위한 커뮤니티 게시판을 붙여, 정형화된 위키와 비정형 대화가 함께 공존하도록 구성했다.

• 기대효과

학사·학교생활 관련 정보를 한 번 검색해 바로 찾을 수 있어 정보 탐색 시간 절약과 학내 생활 편의성 향상을 기대할 수 있다. 정보가 한 곳에 축적될수록 후배 학번이 얻는 이득이 기하급수적으로 커지는 구조이기 때문에, 서비스가 오래 유지될수록 가치가 불어나는 플랫폼이 된다.



사용 기술 및 라이브러리

- **Backend:** Java 17, Spring Boot 3.1.9, Spring MVC, Spring Security, Spring Data JPA, Hibernate, JWT
- **DB:** MySQL

- **Frontend:** React 18, React Query, CKEditor 5, React Router, Axios, STOMP/SockJS
- **Infra:** AWS S3, Docker, docker-compose
- **기타:** Lombok, Spring Validation, JUnit 5, Gradle

역할

6인 팀의 백엔드 2명 중 한 명으로, **회원 인증·계정 복구 전반**과 **예외 처리 체계 확장**을 담당했다.

- **학교 이메일 인증 기반 회원가입 플로우 구현** — Google SMTP를 통해 `@sangmyung.kr` 메일로 인증 요청을 발송하고, 요청·검증의 모든 단계를 DB에 로그로 남겨 재발송·만료·검증 상태를 추적할 수 있도록 설계. 이를 통해 **"재학생만 글을 쓸 수 있다"** 는 서비스 핵심 제약을 백엔드 레벨에서 강제했다.
- **로그인 / 아이디 찾기 / 비밀번호 찾기·변경 API 구현** — 계정 식별 흐름 전반을 담당. 각 단계마다 필요한 검증 로직과 응답 포맷을 정의하고, 보안에 민감한 비밀번호 재설정 경로에는 `BusinessException` 기반 명확한 에러 반환을 적용.
- **에러 코드 체계 확장(`CustomErrorCode` enum)** — 이메일 미존재·인증 실패·재전송 제한 등 인증 도메인 고유의 에러 케이스를 `CustomErrorCode` enum에 정의하고, `@RestControllerAdvice` 기반 `GlobalExceptionHandler` 가 이를 일관된 응답 포맷으로 내려주도록 검증 로직을 추가.
- **JWT 토큰 발급 로직 협업** — 팀 내 다른 백엔드 멤버와 함께 JWT 기반 토큰 발급·검증 구조를 설계·구현. Spring Security의 `SecurityFilterChain` 위에 토큰 기반 인증을 얹고, 프론트엔드가 `@CurrentUser` 로 현재 사용자를 주입받을 수 있는 흐름까지 연결.

깨달은 점

- 외부 연동 작업에서는 "상태 추적"이 기능만큼 중요하다는 걸 배웠다. 처음 이메일 인증을 구현할 때는 "메일만 잘 가면 끝" 이라고 생각했는데, 메일 전송 실패, 만료된 인증 요청 같은 상황들이 서비스의 체감 품질을 결정했다. DB에 인증 요청·검증 로그를 따로 남기기 시작하면서부터 디버깅이 쉬워졌고, 재전송 제한 같은 보조 정책도 이 로그 위에서 자연스럽게 붙일 수 있다는 걸 알게 됐다. 외부 연동(메일·결제·파일 업로드 등)을 다룰 때는 항상 "지금 상태가 어디쯤 와 있는가" 를 DB에서 즉시 답할 수 있어야 한다는 원칙을 이 프로젝트에서 처음 세웠다.
- `GlobalExceptionHandler` + `BusinessException` + `CustomErrorCode` 3요소가 왜 하나의 세트인지 체감했다. 인증 플로우에는 "이미 가입된 이메일", "미존재 계정", "인증 코드 불일치", "만료된 요청" 등 비슷해 보이지만 각기 다른 응답이 필요한 실패 케이스가 유난

히 많다. 이 에러들을 컨트롤러 안에서 직접 처리하려니 코드가 순식간에 지저분해졌는데, `CustomErrorCode` enum에 한 줄만 추가하면 `BusinessException` 하나로 던지고 `@RestControllerAdvice`가 알아서 응답 포맷을 내려주는 구조로 바꾸면서 컨트롤러가 확 가벼워졌다.

- 세션 기반에서 JWT 기반 인증으로 넘어가는 경험은 단순한 기술 교체가 아니었다. React 프론트엔드가 백엔드와 분리되어 있어 상태를 서버가 들고 있을 수 없는 환경이었다. 왜 서버가 상태를 포기하고 토큰을 발급해야 하는지, Spring Security의 필터 체인이 필요한지, 프론트가 매 요청마다 헤더에 토큰을 실어 보내야 하는지를 이 과정에서 비로소 납득했다. 직접 발급 로직 전부를 짜진 않았지만, 팀원과 함께 설계에 참여하면서 "인증 방식의 선택은 아키텍처의 선택"이라는 걸 처음 깨달았다.